

MÉTHODOLOGIE

Ce document explique la démarche derrière ce projet, pas seulement le résultat. Quatre principes reviennent régulièrement au fil du développement.

1. Diagnostiquer par la preuve, jamais par supposition

Chaque bug rencontré a été résolu en observant l'état réel de l'application avant d'agir, jamais en devinant une solution.

CAS RÉEL : Le bouton de refus de permission introuvable

Un test cliquait sur `permission_deny_button`, fonctionnel une première fois puis introuvable une seconde. Plutôt que de supposer un problème de timing, l'écran a été dumpé (uiautomator dump) immédiatement après chaque clic.

La comparaison des XML a révélé que l'identifiant du bouton change selon l'historique des refus : `permission_deny_button` au premier refus, `permission_deny_and_dont_ask_again_button` au second. Ce n'était pas un bug, mais un comportement Android natif depuis l'API 23.

Le test cible désormais le texte visible plutôt qu'un identifiant instable, et gère les deux refus successifs qu'Android impose.

CAS RÉEL : Le crash système en CI après plusieurs sessions

En intégration continue, l'exécution de la suite de tests provoquait un crash du système Android après une douzaine de sessions Appium consécutives (INSTRUMENTATION_ABORTED puis device not found).

La lecture complète des logs CI, plutôt qu'une simple relance, a montré la cause réelle : chaque test relançait une session complète (réinstallation du serveur UiAutomator2, bascule de la politique hidden-api, pm clear), ce qui surchargeait l'émulateur.

La correction a été architecturale, pas cosmétique : une seule session par exécution (fixture pytest scope=session) et une réinitialisation légère de l'état de l'app entre les tests (clearApp puis relance de l'activité), au lieu d'un simple ajustement de timeout.

CAS RÉEL : Le faux positif sur les labels d'accessibilité

Un test vérifiant l'étiquette d'accessibilité des champs de connexion passait au vert alors qu'un relevé manuel de l'arbre UiAutomator2 montrait clairement une étiquette absente.

Le résultat trop propre a été questionné plutôt qu'accepté tel quel : l'assertion comparait la valeur à None ou à une chaîne vide, alors qu'UiAutomator2 renvoie la chaîne littérale 'null' quand l'attribut est absent.

Une fois la comparaison corrigée, le test échoue bien comme attendu, documentant un manquement réel plutôt qu'un faux résultat positif.

CAS RÉEL : Deux comportements de connexion découverts, pas supposés

Avant d'écrire une assertion, chaque comportement suspect a été vérifié manuellement plutôt que déduit de l'interface.

L'application accepte la connexion quel que soit le mot de passe saisi : aucune validation côté client ni serveur n'a pu être mise en évidence.

L'étiquette affichée sur certains comptes est purement visuelle : la connexion aboutit normalement malgré ce libellé, qui ne bloque rien.

Les deux tests documentent ces comportements tels qu'observés, sans supposer qu'il s'agit d'anomalies du côté du test.

Cette méthode s'applique à chaque anomalie du projet : le diagnostic du GPU AMD blacklisté par l'émulateur, le crash de snapshot, ou la redirection du checkout vers le login ont tous été confirmés par preuve avant toute correction.

2. Savoir arrêter, et documenter plutôt que masquer

Le scénario de rotation d'écran programmatique a été tenté selon quatre approches différentes : réglage système dans les deux sens, capability Appium dédiée, version minimale isolée du test. Chacune a échoué avec la même erreur système.

CAS RÉEL : La rotation d'écran

La rotation fonctionne pourtant manuellement, via l'icône de l'interface de l'émulateur. Ça a permis d'isoler la cause réelle : deux mécanismes de rotation distincts existent sur l'émulateur, et seul le mécanisme programmatique échoue sur cette configuration.

Le test est conservé dans le code avec un marqueur skip et une raison explicite plutôt que supprimé ou caché.

CAS RÉEL : Le boot de l'émulateur en intégration continue

Le démarrage de l'émulateur Android en CI échoue de façon intermittente, quelle que soit la configuration retenue : c'est un comportement connu et documenté par la communauté de l'action utilisée (reactivecircus/android-emulator-runner), pas une erreur de configuration de ce projet.

Un mécanisme de reprise à trois tentatives a été ajouté comme filet de sécurité, mais ce n'est pas une correction garantie.

La décision a été de documenter cette limite et de la mettre en pause plutôt que de la masquer derrière un correctif cosmétique qui donnerait une fausse impression de stabilité.

3. Vérifier avant de s'engager, pas après

Avant de choisir l'application cible, sa licence a été vérifiée directement dans le repository plutôt que supposée à partir de sa popularité. Aucun fichier LICENSE n'y est présent, contrairement à d'autres repositories comparables du même éditeur. Ce point est documenté plutôt que passé sous silence, avec la justification du choix malgré cette limite.

4. Étendre la boucle qualité sans gonfler artificiellement le périmètre

L'intégration continue (GitHub Actions), le reporting (Allure) et l'audit d'accessibilité n'ont été ajoutés que parce qu'ils répondent chacun à une vraie question, pas pour élargir la liste des technologies utilisées.

- CI/CD : chaque push exécute la suite de tests sur un émulateur Android hébergé, avec une reprise à trois tentatives pour absorber la flakiness connue du boot d'émulateur (voir principe 2).
- Allure : les résultats bruts de pytest sont transformés en rapport HTML consultable, avec historique d'exécution et détail par test.
- Audit accessibilité : quatre axes vérifiés (étiquettes, taille des zones interactives, contraste, structuration par titres), avec une grille de lecture inspirée du RGAA sans portée légale sur une app native. Le détail complet fait l'objet d'un document séparé.

Sur les sept vérifications d'accessibilité menées, trois documentent un manquement réel. Ce résultat est publié tel quel : l'objectif du projet est de démontrer une méthode d'audit, pas de prétendre que l'application est accessible.

Questions posées avant chaque scénario mobile

Un scénario mobile n'a de valeur que s'il répond à une vraie question de comportement, pas seulement s'il déclenche une action à l'écran.

Permissions runtime : *Que se passe-t-il en cas de refus ? L'application redemande-t-elle, combien de fois ? Et en cas d'acceptation, l'accès est-il immédiat ?*

Background / foreground : *L'état d'un formulaire survit-il à une mise en arrière-plan ? Le panier se comporte-t-il différemment d'un simple champ texte ? Un retour rapide diffère-t-il d'un retour après plusieurs cycles ?*

Interruption système : *Une saisie en cours est-elle perdue si un appel interrompt l'utilisateur ? L'application reprend-elle normalement après ?*

Rotation d'écran : *L'interface reste-t-elle utilisable après un changement d'orientation ? Un champ rempli avant rotation est-il toujours accessible après ?*

Ces questions, plus que le code lui-même, définissent ce qui distingue un test mobile pertinent d'un test qui se contente d'exécuter des actions sans rien valider de réel.

Questions posées avant chaque test d'accessibilité

Étiquettes : *Un lecteur d'écran peut-il identifier ce champ ou ce bouton, ou seulement deviner sa fonction à partir de sa position ?*

Zones interactives : *La zone cliquable est-elle assez grande pour un utilisateur ayant une motricité réduite, indépendamment de la taille visuelle de l'icône ?*

Contraste : *Le texte reste-t-il lisible pour une personne malvoyante, mesuré et non estimé à l'œil ?*

Structuration : *Un utilisateur de lecteur d'écran peut-il sauter directement d'un titre d'écran à l'autre, ou doit-il tout parcourir séquentiellement ?*

Comme pour les scénarios mobiles, chaque test d'accessibilité découle d'une question de comportement réel, pas d'une case à cocher sur une liste de critères.